МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования
«**Московский технический университет связи и информатики**»
(**МТУСИ**)

Кафедра «Системное программирование»

ОТЧЕТ
по лабораторной работе №10
Вариант №26

по дисциплине «**Параллельные и распределенные вычисления**»

Выполнил:
студент гр. БФИ2202
_____ Сидорук Д. В.
« ____ » _____ 2026 г.

Проверил:
старший преподаватель
_____ Клешнин Н. Г.
ассистент
_____ Соловьев А. С.
« ____ » _____ 2026 г.

Москва, 2026 г.

Содержание

1 Введение 3
2 Цель работы 3
3 Задание 3
4 Ход работы 3
Заключение 10

1 Введение

В данной лабораторной работе рассматривается реализация блочного алгоритма свёртки изображения на графическом процессоре с использованием технологии CUDA. Актуальность темы обусловлена широким применением операции свёртки в задачах цифровой обработки изображений, компьютерного зрения, распознавания образов и подготовки данных для последующего анализа. Свёртка используется для выделения контуров, сглаживания изображений, повышения резкости и выполнения других преобразований, необходимых в современных вычислительных системах.

Целью лабораторной работы является изучение принципов организации параллельных вычислений при выполнении двумерной свёртки изображения с учётом цветовых каналов, а также освоение методов оптимизации доступа к памяти GPU. В работе требуется реализовать алгоритм свёртки с маской размером 5×5, используя разделяемую память для сокращения числа обращений к глобальной памяти и константную память для хранения маски свёртки. Такой подход позволяет повысить эффективность выполнения ядра и лучше понять особенности архитектуры графических ускорителей.

Особенностью рассматриваемой задачи является обработка перемежёванного изображения в формате RGB, где каждый пиксель представлен тремя цветовыми компонентами. Для каждого выходного пикселя и каждого канала необходимо вычислить сумму произведений значений соседних пикселей на соответствующие коэффициенты маски, с последующим ограничением результата в диапазоне от 0 до 1. Кроме того, в реализации необходимо корректно учитывать граничные условия, возникающие при обработке пикселей, расположенных у краёв изображения.

Выполнение лабораторной работы направлено на закрепление навыков разработки CUDA-программ, изучение особенностей использования глобальной, разделяемой и константной памяти, а также анализ влияния структуры алгоритма на производительность вычислений. Полученные результаты позволяют оценить преимущества применения GPU для решения задач свёрточной обработки изображений.

2 Цель работы

Получение навыков вычисления свертки изображения.

3 Задание

Реализовать программу, реализующую блочный алгоритм свертки изображения, испольщующий как разделяемую, так и константную память, с помощью CUDA.

4 Ход работы

В листинге ниже приведено содержание файла rvs_laboratories_9/rvs_laboratories_9.c который содержит код, выполняющий свертку изображения на GPU с помощью CUDA (лист. 1).

Лист. 1 – Содержание rvs_laboratories_9/rvs_laboratories_9.cu

```
#include "wb.h"

#define wbCheck(stmt) \
  do { \
    cudaError_t err = stmt; \
    if (err != cudaSuccess) { \
      wbLog(ERROR, "Failed to run stmt ", #stmt); \
      return -1; \
    } \
  } while (0)

#define Mask_width 5
#define Mask_radius Mask_width / 2
#define TILE_WIDTH 16
#define w (TILE_WIDTH + Mask_width - 1)
#define clamp(x) (min(max((x), 0.0), 1.0))

__constant__ float constMask[Mask_width * Mask_width];

__global__ void convolution(const float *inputImage, const float *mask,
                            float *outputImage, int channels,
                            int width, int height) {
  __shared__ float tile[w][w][3];

  int tx = threadIdx.x;
  int ty = threadIdx.y;

  int row_o = blockIdx.y * TILE_WIDTH + ty;
  int col_o = blockIdx.x * TILE_WIDTH + tx;

  int row_i = row_o - Mask_radius;
  int col_i = col_o - Mask_radius;

  for (int c = 0; c < channels; ++c) {
    if (row_i >= 0 && row_i < height && col_i >= 0 && col_i < width) {
      tile[ty][tx][c] = inputImage[(row_i * width + col_i) * channels + c];
    } else {
      tile[ty][tx][c] = 0.0f;
    }
  }

  __syncthreads();

```

```
  if (tx < TILE_WIDTH && ty < TILE_WIDTH &&
      row_o < height && col_o < width) {
    for (int c = 0; c < channels; ++c) {
      float accum = 0.0f;
      for (int i = 0; i < Mask_width; ++i) {
        for (int j = 0; j < Mask_width; ++j) {
          accum += tile[ty + i][tx + j][c] * constMask[i * Mask_width + j];
        }
      }
      outputImage[(row_o * width + col_o) * channels + c] = clamp(accum);
    }
  }
}

int main(int argc, char *argv[]) {
  wbArg_t arg;
  int maskRows;
  int maskColumns;
  int imageChannels;
  int imageWidth;
  int imageHeight;
  char *inputImageFile;
  char *inputMaskFile;
  wbImage_t inputImage;
  wbImage_t outputImage;
  float *hostInputImageData;
  float *hostOutputImageData;
  float *hostMaskData;
  float *deviceInputImageData;
  float *deviceOutputImageData;
  float *deviceMaskData;

  arg = wbArg_read(argc, argv);

  inputImageFile = wbArg_getInputFile(arg, 0);
  inputMaskFile  = wbArg_getInputFile(arg, 1);

  inputImage   = wbImport(inputImageFile);
  hostMaskData = (float *)wbImport(inputMaskFile, &maskRows, &maskColumns);

  assert(maskRows == 5);
  assert(maskColumns == 5);

  imageWidth    = wbImage_getWidth(inputImage);
  imageHeight   = wbImage_getHeight(inputImage);
  imageChannels = wbImage_getChannels(inputImage);

  outputImage = wbImage_new(imageWidth, imageHeight, imageChannels);

  hostInputImageData  = wbImage_getData(inputImage);
```

```
hostOutputImageData = wbImage_getData(outputImage);

wbTime_start(GPU, "Doing GPU Computation (memory + compute)");

wbTime_start(GPU, "Doing GPU memory allocation");
wbCheck(cudaMalloc((void **)&deviceInputImageData,
                   imageWidth * imageHeight * imageChannels * sizeof(float)));
wbCheck(cudaMalloc((void **)&deviceOutputImageData,
                   imageWidth * imageHeight * imageChannels * sizeof(float)));
wbCheck(cudaMalloc((void **)&deviceMaskData,
                   maskRows * maskColumns * sizeof(float)));
wbTime_stop(GPU, "Doing GPU memory allocation");

wbTime_start(Copy, "Copying data to the GPU");
wbCheck(cudaMemcpy(deviceInputImageData, hostInputImageData,
                   imageWidth * imageHeight * imageChannels * sizeof(float),
                   cudaMemcpyHostToDevice));
wbCheck(cudaMemcpy(deviceMaskData, hostMaskData,
                   maskRows * maskColumns * sizeof(float),
                   cudaMemcpyHostToDevice));
wbCheck(cudaMemcpyToSymbol(constMask, hostMaskData,
                           maskRows * maskColumns * sizeof(float)));
wbTime_stop(Copy, "Copying data to the GPU");

wbTime_start(Compute, "Doing the computation on the GPU");
dim3 dimBlock(w, w, 1);
dim3 dimGrid((imageWidth + TILE_WIDTH - 1) / TILE_WIDTH,
             (imageHeight + TILE_WIDTH - 1) / TILE_WIDTH, 1);
convolution<<<dimGrid, dimBlock>>>(deviceInputImageData, deviceMaskData,
                                   deviceOutputImageData, imageChannels,
                                   imageWidth, imageHeight);
wbCheck(cudaDeviceSynchronize());
wbTime_stop(Compute, "Doing the computation on the GPU");

wbTime_start(Copy, "Copying data from the GPU");
wbCheck(cudaMemcpy(hostOutputImageData, deviceOutputImageData,
                   imageWidth * imageHeight * imageChannels * sizeof(float),
                   cudaMemcpyDeviceToHost));
wbTime_stop(Copy, "Copying data from the GPU");

wbTime_stop(GPU, "Doing GPU Computation (memory + compute)");

wbSolution(arg, outputImage);

wbCheck(cudaFree(deviceInputImageData));
wbCheck(cudaFree(deviceOutputImageData));
wbCheck(cudaFree(deviceMaskData));
```

```

  free(hostMaskData);
  wbImage_delete(outputImage);
  wbImage_delete(inputImage);

  return 0;
}
```

В листинге ниже приведено содержание файла CMakeLists.txt, определяющего правила сборки программы (лист. 2).

Лист. 2 – Содержание CMakeLists.txt

```
cmake_minimum_required(VERSION 3.27)
project(rvs_laboratories_9 LANGUAGES CXX CUDA)

set(LIBGPUTK_PATH "/usr/local/libgputk" CACHE STRING "Path to
 ↪  libgputk")

add_executable(dataset_generator
 ↪  dataset_generator/dataset_generator.cpp)

target_include_directories(dataset_generator PRIVATE
 ↪  ${LIBGPUTK_PATH}/include)
target_link_directories(dataset_generator PRIVATE
 ↪  ${LIBGPUTK_PATH}/lib/)
target_link_libraries(dataset_generator gputk)

add_executable(rvs_laboratories_9
 ↪  rvs_laboratories_9/rvs_laboratories_9.cu)
```

Сгенерируем тестовые входные данные и проверим, что они действительно были сгенерированы (рис. 1).

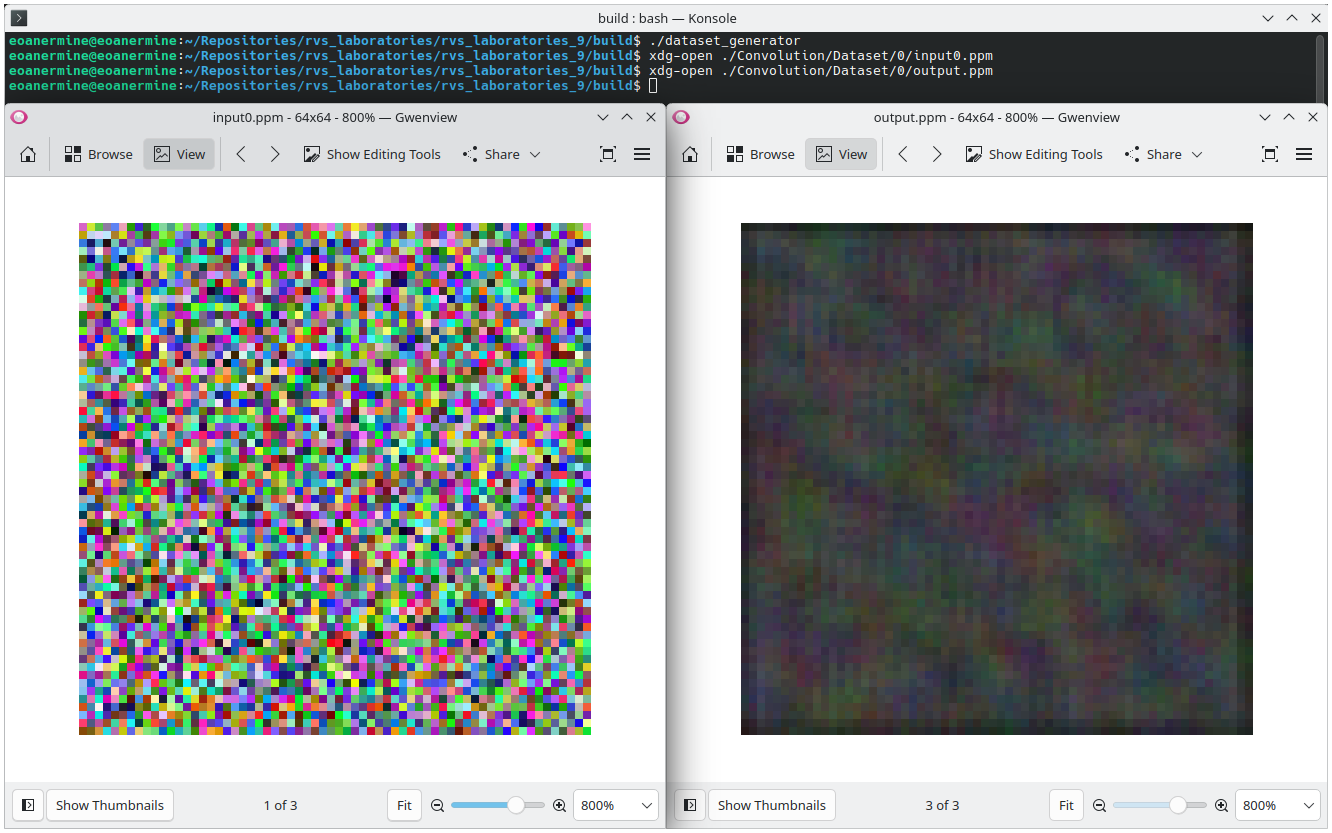

Рис. 1 – Генерация тестовых входных данных

На рисунке ниже приведен результат сборки и выполнения программы (лист. 2)

```
build : bash — Konsole
eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_9/build$ cmake ..
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: /home/eoanermine/Repositories/rvs_laboratories/rvs_laboratories_9/build
eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_9/build$ make
[ 50%] Built target dataset_generator
[100%] Built target rvs_laboratories_9
eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_9/build$ ./rvs_laboratories_9 Convolution/Dataset/0/input0.ppm Convolution/Dataset/0/input1
.raw Convolution/Dataset/0/output.ppm
[GPU    ] 0.000237824 Doing GPU memory allocation
[Copy   ] 0.000947968 Copying data to the GPU
[Compute] 0.000346880 Doing the computation on the GPU
[Copy   ] 0.000038912 Copying data from the GPU
[GPU    ] 0.001627136 Doing GPU Computation (memory + compute)
Solution is correct.
eoanermine@eoanermine:~/Repositories/rvs_laboratories/rvs_laboratories_9/build$
```

Рис. 2 – Компиляция и выполнение программы

Как можно видеть, вычисление свертки было выполнено успешно.

На рисунке ниже приведено изображение, созданное в результате выполнения программы.

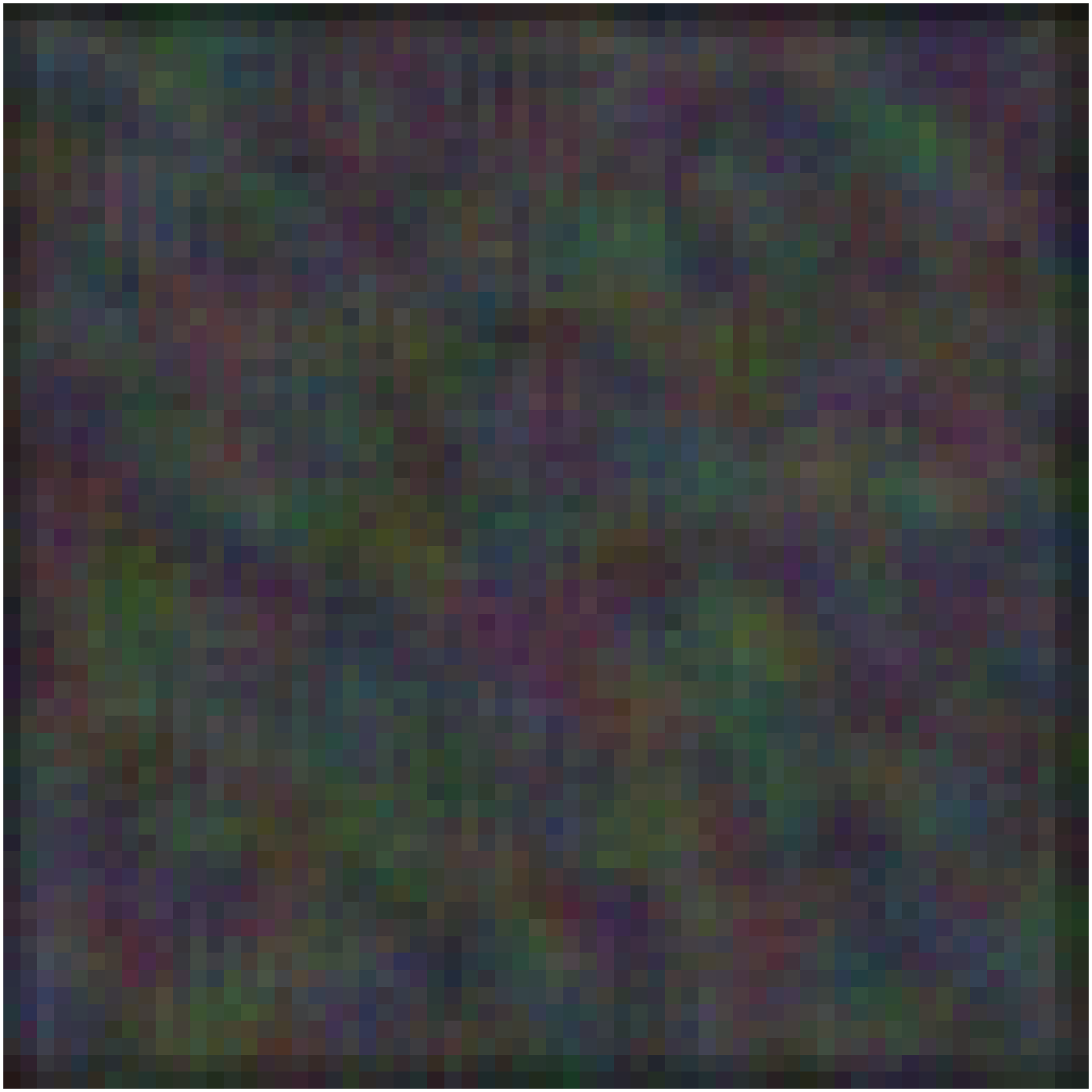

Рис. 3 – Выходное изображение

Как видно, оно совпадает с эталонным изображением.

Заключение

Таким образом, в ходе выполнения данной лабораторной работы были успешно получены навыки вычисления свертки изображения. На практике была реализована программа, реализующая блочный алгоритм свертки изображения, испольщующий как разделяемую, так и константную память, с помощью CUDA.